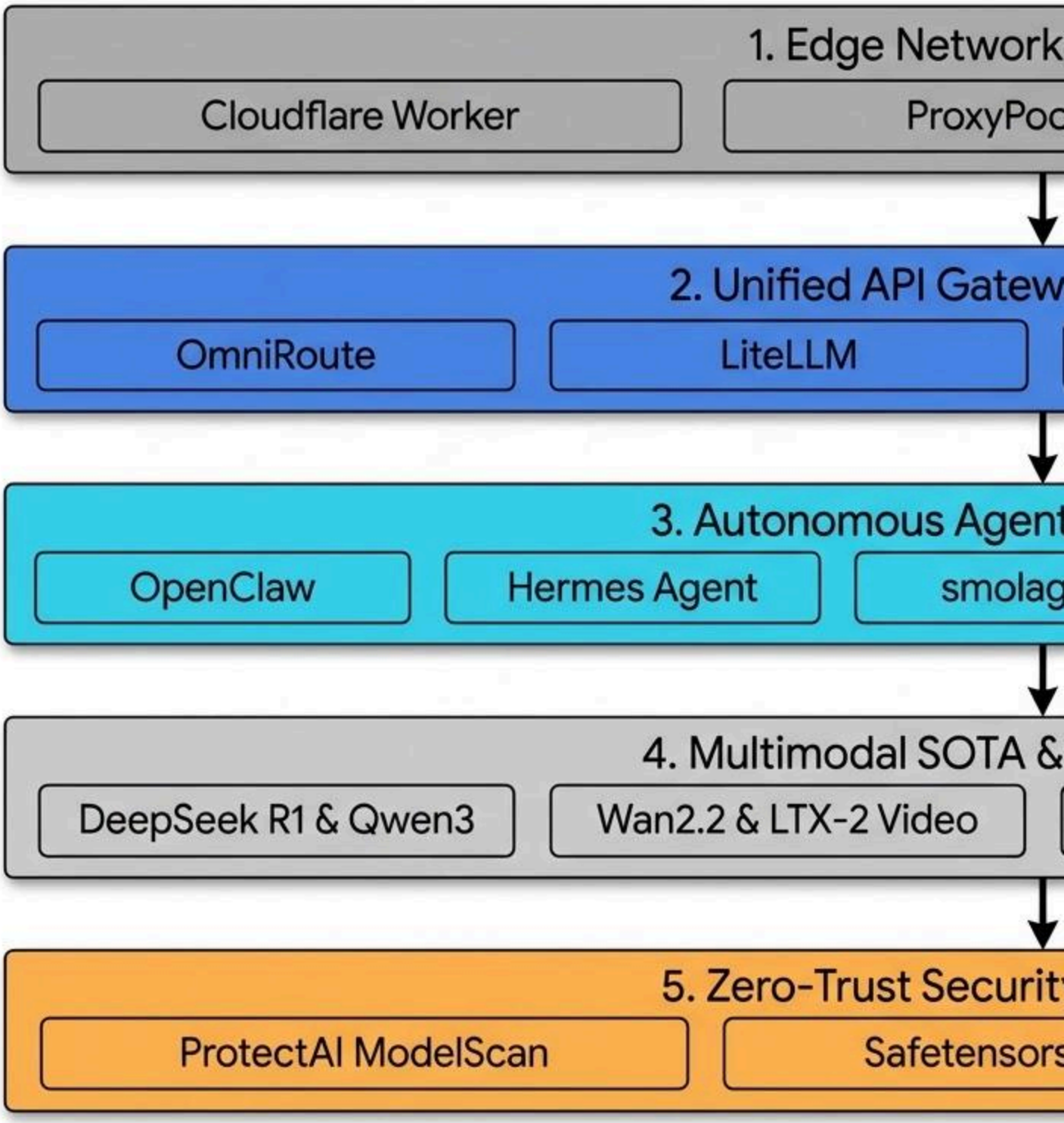
GPU khusus:

1. **Bittensor Monorepo & SDK (subtensor & btcli):** https://github.com/RaoFoundation/bittensor — Menggunakan dompet kripto lokal (btcli wallet new_coldkey).
2. **Bittensor Subnet Template:** https://github.com/opentensor/bittensor-subnet-template — Kerangka dasar pembuatan node *miner* dan *validator*.
3. **Panduan Eksekusi Testnet Bittensor (Faucet via Discord):** https://github.com/opentensor/bittensor-subnet-template/blob/main/docs/running_on_testnet.md
4. **Spesifikasi Perangkat Keras Minimum Miner Bittensor:** https://github.com/opentensor/bittensor-subnet-template/blob/main/min_compute.yml — Menetapkan syarat minimum GPU (seperti 24 GB VRAM untuk subnet komputasi).
5. **Subnet 27 Compute Bittensor (commune-ai/compute-ni):** https://github.com/commune-ai/compute-ni — Memberikan insentif TAO atas penyediaan GPU berdasarkan rumus skor performa.
6. **Score Predict Subnet Bittensor (Mining Prediksi Olahraga):** https://github.com/score-technologies/score-predict/blob/main/README.md — Menggunakan model *Random Forest* dan dompet Bittensor.
7. **Nosana CLI (Node GPU Terdesentralisasi di Jaringan Solana):** https://github.com/nosana-ci/nosana-cli — Wajib dompet kripto Solana (SOL / NOS).
8. **Nosana LLM Benchmarking Suite:** https://github.com/nosana-ci/nosana-llm-benchmarking — Pengujian performa GPU untuk kualifikasi node Nosana.
9. **Portal Jaringan Nosana:** https://github.com/nosana-ci/nosana.io
10. **IO.net Launch Binaries (Worker Klaster GPU Terdesentralisasi):** https://github.com/ionet-official/io_launch_binaries/issues/48 — Wajib dompet Solana untuk menerima distribusi insentif/airdrop.

Arsitektur Frontier Agregator AI Agentik Otonom Tanpa Biaya: Sintesis Ekosistem Kode Terbuka, Model Tanpa Sensor, dan Infrastruktur Terdesentralisasi

Pembangunan agregator kecerdasan buatan (AI) agentik yang sepenuhnya otonom, bebas biaya operasional (*zero-cost*), mendukung eksekusi model tanpa sensor (*uncensored/unfiltered*), serta terjamin keamanannya dari ancaman kode berbahaya (*malware*) menuntut konvergensi lintas disiplin antara rekayasa sistem terdistribusi, keamanan kriptografi bobot model, dan orkestrasi jaringan terdesentralisasi. Dalam lanskap komputasi modern, ketergantungan pada satu penyedia layanan komersial menciptakan titik kegagalan tunggal (*single point of failure*), pembatasan kuota yang kaku, serta penyensoran heuristik yang sering kali menghambat penelitian akademik dan eksplorasi teknis tingkat lanjut. Penelitian ini membedah secara komprehensif ekosistem repositori kode terbuka di GitHub dan Hugging Face untuk merumuskan cetak biru arsitektur agregator AI otonom yang mengintegrasikan sumber daya komputasi gratis, perutean pintar (*smart routing*), manipulasi ruang aktivasi residual (*abliteration*), model *State-of-the-Art* (SOTA) multimodal, serta infrastruktur jaringan proksi anonim.



Paradigma Ekonomi Komputasi: Strategi Airdrop, Bantuan Sosial Komputasi, dan Jaringan DePIN

Membangun agregator AI berskala penuh tanpa mengeluarkan modal finansial memerlukan eksploitasi sistematis terhadap dua fenomena utama dalam ekosistem teknologi terbuka: agregasi kuota "bantuan sosial" (*bansos*) komputasi—yaitu istilah komunitas pengembang untuk distribusi kuota API gratis, *free-tier* layanan *cloud*, serta stasiun relai publik nirlaba (*public charity stations* atau 公益站)—dan partisipasi dalam jaringan *Decentralized Physical Infrastructure Networks* (DePIN) yang memberikan insentif token kripto maupun *airdrop* atas kontribusi sumber daya komputasi. Analisis terhadap repositori GitHub menunjukkan bahwa kelangkaan komputasi bagi peneliti independen telah diatasi melalui penggabungan ratusan titik akhir (*endpoints*) gratis ke dalam satu antarmuka terpadu.

Pada lapisan agregasi API gratis, proyek seperti tashfeenahmed/freellmapi menyediakan akses ke 7,4 miliar token per bulan yang dikonsolidasikan dari 34 penyedia LLM gratis dan 635 titik akhir model di balik satu antarmuka /v1 tunggal. Lebih jauh lagi, diegosouzapw/OmniRoute berfungsi sebagai *gateway* AI berlisensi MIT yang mengintegrasikan 359 penyedia layanan—di mana lebih dari 150 di antaranya sepenuhnya gratis—mencakup lebih dari 1.200 model termasuk keluarga Kimi, Claude, GPT, dan Gemini agar proses pemrograman dan eksekusi agen tidak pernah terhenti akibat batas kuota. Untuk kebutuhan pengembangan kode secara spesifik, decolua/9router menyediakan jembatan penghubung tanpa batas (*unlimited free AI coding*) yang menghubungkan agen pemrograman seperti Claude Code, Codex, Cursor, Cline, dan Copilot langsung ke titik akhir gratis Claude, GPT, dan Gemini. Selain itu, repositori seperti justlovemaki/AIClient2API dan edison7009/EchoBird memungkinkan konversi akun langganan antarmuka klien atau CLI (seperti Antigravity, Codex CLI, Kiro, Grok, dan Claude Code) menjadi proksi API multi-protokol mandiri (*self-hosted*) dengan pelacakan kuota terpusat.

Di luar titik akhir inferensi model, kebutuhan infrastruktur dasar seperti *storage*, *hosting*, basis data, *domain*, dan CI/CD dikatalogkan secara menyeluruh dalam repositori ripienaar/free-for-dev beserta antarmuka pencariannya di n8watkins/freestack yang mengindeks lebih dari 1.200 layanan *free-tier* untuk pengembang. Direktori ini mencakup paket penyimpanan data gratis (seperti alokasi penyimpanan data 100 MB, penyimpanan berkas 1 GB, dan *bandwidth* 5 GB pada tingkat layanan mikro) serta daftar kolektif API terbuka dari public-apis/public-apis. Dalam ekosistem komunitas Asia Timur yang menggunakan songquanpeng/one-api, pengembang juga secara aktif berbagi stasiun relai API gratis (公益站) melalui forum diskusi dan *issues* GitHub untuk mendistribusikan kelebihan kuota komputasi secara kolektif.

Mekanisme Insentif Jaringan Komputasi Terdesentralisasi (DePIN) dan Airdrop

Bagi arsitek sistem yang ingin menghasilkan nilai ekonomi atau memperoleh kredit komputasi secara mandiri, ekosistem *blockchain* AI dan DePIN menawarkan mekanisme *mining* dan *validation* yang terukur secara matematis. Protokol utama dalam kategori ini adalah Bittensor (opentensor/bittensor), sebuah jaringan saraf berskala internet di mana satu *blockchain* utama bernama subtensor mengoordinasikan berbagai pasar komoditas digital independen yang disebut *subnet*. Sejak rilis Bittensor 11, pengembangan SDK Python, alat baris perintah btcli, dan manajemen dompet kripto (coldkey untuk penyimpanan nilai dan hotkey untuk operasional node aktif) telah dikonsolidasikan ke dalam monorepo subtensor (sdk/python), menggantikan tumpukan jaringan lama berbasis Axon/Dendrite/Synapse pada versi 10 dengan lapisan identitas berbasis permintaan tertanda tangan (*signed requests*).

Dalam ekosistem Bittensor, peserta dapat menjalankan node *miner* yang memproduksi

komoditas digital—mulai dari inferensi teks (Subnet 1), daya komputasi mentah (Subnet 15 di *testnet* atau Subnet 27 di *mainnet* melalui commune-ai/compute-ni / neuralinternet/Compute-Subnet), prediksi pasar, hingga pemodelan prediktif olahraga (score-technologies/score-predict)—dan mendapatkan imbalan dalam bentuk token TAO berdasarkan evaluasi dari *validator* melalui mesin konsensus **Yuma Consensus**. Untuk mendaftarkan *subnet* atau berpartisipasi di jaringan uji (*testnet*), operator membuat pasangan kunci melalui btcli wallet new_coldkey dan btcli wallet new_hotkey, memeriksa biaya penguncian slot melalui btcli subnet lock_cost --subtensor.network test (dengan biaya dasar \tau 100), serta memperoleh token uji gratis melalui komunitas Discord Bittensor karena fitur *faucet* otomatis dinonaktifkan.
Penilaian kontribusi perangkat keras pada Subnet Komputasi Bittensor (netuid 27) dihitung menggunakan fungsi penskalaan deterministik yang menggabungkan kinerja GPU dan bonus alokasi sumber daya. Berdasarkan spesifikasi protokol di commune-ai/compute-ni, skor total sebuah node penambang (S_{\text{total}}) dirumuskan sebagai berikut:
S_{\text{GPU}} = \text{Perf}_{\text{base}} \times N_{\text{GPU}} \times \left(\frac{100}{\text{Max_Scenario}}\right) S_{\text{total}} = S_{\text{GPU}} + B_{\text{alloc}}
Di mana \text{Perf}_{\text{base}} adalah metrik kinerja dasar unit GPU, N_{\text{GPU}} adalah jumlah unit pemrosesan grafis yang disediakan, dan \text{Max_Scenario} adalah batas skenario tertinggi (misalnya 3,90 \times 8 = 31,2, yang menghasilkan faktor pengali \approx 3,2051 sehingga skor maksimum GPU mencapai 100). Variabel B_{\text{alloc}} merupakan bonus alokasi sebesar 100 poin yang diberikan apabila sumber daya mesin penambang sedang dialokasikan secara aktif kepada validator melalui skrip register.py, sehingga skor maksimum mencapai 200 poin. Demikian pula, pada *subnet* berbasis akurasi prediksi seperti score-technologies/score-predict, imbalan penambang yang menjalankan pengklasifikasi *Random Forest* dikalikan dengan faktor pengganda beruntun (*streak multiplier* hingga 1,8\times untuk \ge 20 prediksi benar berturut-turut) dan pengganda tingkat kemenangan 7 hari (*win rate multiplier* sebesar 1,3\times untuk akurasi \ge 70\%, atau penalti 0,9\times jika di bawah 40\%).
Selain Bittensor, ekosistem berbasis *blockchain* Solana melalui **Nosana** (nosana-ci/nosana-cli dan nosana-ci/nosana-llm-benchmarking) memungkinkan pemilik GPU menganggur (*idle GPU*) untuk menjalankan *Nosana Node* guna memproses beban kerja inferensi AI dan jalur CI/CD dengan imbalan token jaringan, sekaligus menyediakan komputasi *cloud* berbiaya sangat rendah bagi pengembang. Paralel dengan itu, jaringan **IO.net** (ionet-official/io_launch_binaries) menyediakan alat baris perintah *IO Worker* untuk mengagregasi GPU terdistribusi ke dalam klaster komputasi terdesentralisasi.

Kategori Sumber Daya	Nama Proyek / Repositori	Kapasitas Gratis / Mekanisme Insentif	Spesifikasi Teknis & Protokol
Agregator API Gratis	diegosouzapw/OmniRoute	150+ penyedia gratis dari total 359 penyedia; 1.200+ model (Kimi, Claude, GPT, Gemini)	Lisensi MIT, ditulis dalam TypeScript, satu titik akhir terpadu
Agregator API Gratis	tashfeenahmed/freellmapi	7,4 miliar token/bulan; 34 penyedia LLM gratis; 635 titik akhir model gratis	Antarmuka kompatibel /v1 tunggal, ditulis dalam TypeScript
Router Koding Gratis	decolua/9router	Koding AI gratis tanpa batas untuk Claude, GPT, dan Gemini	Kompatibel dengan Claude Code, Codex, Cursor, Cline, Copilot,

Kategori Sumber Daya	Nama Proyek / Repositori	Kapasitas Gratis / Mekanisme Insentif	Spesifikasi Teknis & Protokol
			Antigravity
Proksi Akun & Kuota	justlovemaki/AIClient2API	Konversi klien gratis/langganan (Antigravity, Codex, Grok, Kiro, Claude) ke API	Proksi mandiri multi-protokol berbasis JavaScript
Manajemen Kuota CLI	edison7009/EchoBird	Pelacakan kuota terpusat untuk akun ChatGPT, Codex CLI, dan Claude Code	Ditulis dalam Rust dengan fitur *multi-model smart routing*
Katalog Infrastruktur Gratis	ripienaar/free-for-dev & n8watkins/freestack	1.200+ layanan *free-tier* untuk *hosting*, basis data, *storage*, CI/CD, dan API	Mencakup alokasi mikro (100 MB DB, 1 GB berkas, 5 GB *bandwidth*)
Komputasi DePIN & Mining	opentensor/bittensor & commune-ai/compute-ni	Imbalan token TAO melalui Yuma Consensus atas kontribusi model AI atau GPU	Spesifikasi minimum *miner*: 4-8 core CPU (2,5-3,5 GHz), 16 GB RAM, 24 GB VRAM (A100), SSD 100 GB (5.000 IOPS)
Komputasi DePIN GPU	nosana-ci/nosana-cli & ionet-official/io_launch_binaries	Monetisasi GPU menganggur melalui *Nosana Node* (Solana) dan *IO Worker*	Mendukung *benchmarking* LLM dan eksekusi kontainer terdesentralisasi

Arsitektur Router, Gateway API, dan Orkestrasi Lalu Lintas Multi-Protokol

Inti dari agregator AI otonom adalah lapisan *API Gateway* dan *Model Router* yang bertugas menerima permintaan dari agen, memilih model yang paling optimal berdasarkan kompleksitas tugas dan ketersediaan kuota gratis, serta menangani kegagalan jaringan (*failover*) secara transparan. Tanpa lapisan perutean yang cerdas, sistem otonom akan cepat kehabisan kuota pada model *frontier* atau mengalami kegagalan eksekusi akibat pembatasan laju permintaan (*rate limiting*) dari penyedia hulu.

Ekosistem Gateway Terpadu: One-API, New-API, dan LiteLLM

Dalam arsitektur *gateway* mandiri (*self-hosted*), songquanpeng/one-api (37,2 ribu bintang di GitHub) telah menjadi fondasi standar untuk mendistribusikan berbagai model besar ke dalam format panggilan standar OpenAI. Sistem ini dikemas sebagai *binary* tunggal atau kontainer Docker (justsong/one-api atau ghcr.io/songquanpeng/one-api) yang mendukung penyebaran multi-mesin (*multi-node deployment*). Dalam konfigurasi klaster berkapasitas tinggi, seluruh server wajib berbagi nilai SESSION_SECRET yang identik, terhubung ke basis data MySQL terpusat melalui SQL_DSN (menggantikan SQLite bawaan), mengaktifkan *caching* melalui REDIS_CONN_STRING untuk mencapai nol akses basis data saat *cache* masih berlaku, serta

menetapkan NODE_TYPE=slave pada node pekerja dan mengatur SYNC_FREQUENCY untuk sinkronisasi konfigurasi berkala.
Sebagai pengembangan sekunder dari One-API, Calcium-Ion/new-api (beserta proyek turunannya seperti lachlanchen/LazyRouter) memperluas dukungan protokol secara signifikan dengan mengintegrasikan Midjourney Proxy, pembuatan musik Suno, model *Rerank*, antarmuka pesan asli Anthropic (/v1/messages), serta dukungan ekosistem alat tambahan seperti Calcium-Ion/new-api-key-tool untuk pelacakan kuota berbasis web. Untuk mengatasi lonjakan beban CPU dan memori pada skenario konkurensi ekstrem dan percobaan ulang (*retries*) yang tinggi, varian Calcium-Ion/new-api-horizon (dirilis hingga versi 0.4.24-alpha.1 melalui citra Docker calciumion/new-api-horizon:latest) melakukan refaktorisasi logika internal yang menghemat sedikitnya 50% konsumsi CPU dan memori pada ukuran *payload* besar, serta menurunkan beban CPU sebesar 5% tambahan melalui fitur optimasi mode *streaming* (*Server-Sent Events* / SSE).
Di sisi lain, BerriAI/litellm (61,4 ribu bintang) menawarkan arsitektur berbasis inti Rust (*Rust core*) dengan SDK Python yang mampu memanggil lebih dari 100 API LLM (termasuk Bedrock, Azure, OpenAI, Anthropic, VertexAI, vLLM, dan Nvidia NIM) dengan pelacakan biaya, *guardrails*, dan penyeimbangan beban (*load balancing*). Ekosistem *gateway* modern juga telah berevolusi untuk mendukung *Model Context Protocol* (MCP) secara langsung, sebagaimana terlihat pada maximhq/bifrost (gateway Go berkinerja tinggi untuk LLM, MCP, dan agen), agentgateway/agentgateway (proksi agentik generasi berikutnya berbasis Rust untuk agen AI dan server MCP), TykTechnologies/tyk, serta oomol-lab/open-connector yang menghubungkan lebih dari 1.500 penyedia SaaS ke agen AI melalui antarmuka SDK, CLI, MCP, HTTP, dan OpenAPI. Untuk efisiensi konteks pada agen koding, mksglu/context-mode menyediakan mekanisme *sandboxing* keluaran alat yang mampu memangkas ukuran token konteks hingga 98% sambil mempertahankan memori sesi.

Perutean Pintar Berbasis Pembelajaran Mesin (RouteLLM dan Router Agentik)

Menggabungkan banyak model saja tidak cukup; sebuah agregator otonom memerlukan pengklasifikasi kueri waktu nyata (*real-time query router*) yang mampu membedakan antara instruksi sederhana yang dapat diselesaikan oleh model lokal atau gratis berukuran kecil dan instruksi penalaran kompleks yang membutuhkan model SOTA berkapasitas besar. Landasan akademik dari mekanisme ini dikembangkan oleh peneliti dari Stanford University melalui *FrugalGPT*, diikuti oleh konsorsium Martian, UC Berkeley, dan UC San Diego melalui *RouterBench* (withmartian/routerbench), serta mencapai kematangan implementasi pada kerangka kerja **RouteLLM** (lm-sys/RouteLLM) yang dikembangkan oleh UC Berkeley, Anyscale, dan Canva.
Kerangka kerja lm-sys/RouteLLM menyediakan empat arsitektur perute bawaan yang dilatih menggunakan data preferensi manusia dari *Chatbot Arena* yang telah diperkaya dengan penilaian GPT-4 (*augmented preference data*, tersedia di Hugging Face sebagai routellm/mf_gpt4_augmented). Arsitektur pertama dan yang paling direkomendasikan untuk lingkungan produksi adalah **Matrix Factorization (mf)**, yang menggunakan model faktorisasi matriks guna memproyeksikan representasi laten dari kueri pengguna dan kapabilitas model guna memprediksi probabilitas kemenangan model kuat terhadap model lemah; metode ini memiliki daya generalisasi yang sangat tinggi sehingga pasangan LLM dapat diganti tanpa perlu melatih ulang perute. Arsitektur kedua adalah **Similarity-Weighted Elo (sw_ranking)**,

yang menghitung peringkat Elo tertimbang secara dinamis di mana setiap suara historis diberi bobot berdasarkan kemiripan semantik (*embedding similarity*) dengan prompt pengguna saat ini. Arsitektur ketiga menggunakan **BERT Classifier (bert)**, yakni pengklasifikasi berbasis transformator dua arah yang dilatih langsung pada data preferensi untuk memprediksi kebutuhan rute kueri, sedangkan arsitektur keempat menerapkan **Causal LLM Classifier (causal_llm)** yang menyetel secara khusus (*fine-tuned*) model bahasa kausal pada data preferensi instruksi.
Secara matematis, RouteLLM mengendalikan kompromi (*trade-off*) antara kualitas respons dan biaya komputasi melalui parameter ambang batas (\tau). Untuk setiap prompt masukan x, perute menghitung skor probabilitas P(\text{strong} \succ \text{weak} \mid x) dan menerapkan aturan keputusan:
\text{Route}(x) = \begin{cases} \mathcal{M}_{\text{strong}} & \text{jika } P(\text{strong} \succ \text{weak} \mid x) \ge \tau \\ \mathcal{M}_{\text{weak}} & \text{jika } P(\text{strong} \succ \text{weak} \mid x) < \tau \end{cases}
Dalam kalibrasi standar menggunakan distribusi kueri Chatbot Arena pada perute mf, nilai ambang batas \tau = 0,11593 menghasilkan alokasi tepat 50% kueri tersulit ke model kuat (\mathcal{M}_{\text{strong}}) dan 50% sisanya ke model lokal seperti ollama_chat/llama3 yang berjalan pada http://localhost:11434/v1. Selain RouteLLM, perute khusus sistem agentik seperti weave-os/router (ditulis dalam Go) mampu merutekan setiap prompt ke model yang tepat dalam waktu kurang dari 50 milidetik dan memangkas biaya operasional sebesar 40% hingga 70%, sedangkan BlockRunAI/ClawRouter menghadirkan perutean lokal di bawah 1 milidetik (<1ms local routing) yang dirancang khusus untuk agen otonom. Untuk perangkat dengan keterbatasan memori fisik yang ekstrem, inovasi dari Edge0-AI/Edge0 menggunakan perute terlatih yang memprediksi aktivasi *expert* satu langkah di depan (*step-ahead router prediction*), sehingga memungkinkan eksekusi model *Mixture-of-Experts* (MoE) berukuran 35 miliar parameter hanya dalam \sim 2,5\text{ GB} RAM dengan mengalirkan bobot (*weight streaming*) langsung dari penyimpanan SSD.

Mitigasi Kerentanan Keamanan pada Lapisan Gateway

Implementasi *gateway* kode terbuka pada lingkungan produksi menuntut audit keamanan terhadap celah arsitektural yang telah terdokumentasi di repositori hulu. Evaluasi teknis terhadap daftar masalah (*issues*) pada songquanpeng/one-api dan Calcium-Ion/new-api mengungkap beberapa vektor risiko kritis yang wajib ditambal oleh arsitek sistem. Risiko pertama berkaitan dengan kondisi balapan kuota (*Time-of-Check to Time-of-Use* / TOCTOU *race condition*) sebagaimana tercatat pada laporan one-api#2440, di mana permintaan konkuren secara simultan dapat mengeksploitasi jeda waktu validasi untuk melampaui batas kuota token yang ditetapkan. Risiko kedua melibatkan eskalasi hak akses dan otorisasi kanal, di mana laporan one-api#2423 dan one-api#2425 mengidentifikasi kerentanan akses kanal tanpa otorisasi melalui manipulasi parameter URL channelid (CWE-863) serta potensi kebocoran token akses *root* oleh akun tingkat administrator, di samping kewajiban mutlak untuk segera mengganti kata sandi bawaan akun root (123456) saat inisialisasi pertama.
Lebih lanjut, pada aspek integritas protokol, laporan one-api#2458 mengungkap anomali injeksi prompt sistem pada protokol Gemini, yakni cacat translasi di mana prompt sistem (*system prompt*) dikirim sebagai giliran pengguna (*user turn*) yang diikuti oleh respons model palsu berbunyi "Okay" karena daftar izin (allow-list) untuk parameter systemInstruction hanya memuat tiga model seri gemini-2.0. Cacat ini dapat mendistorsi perilaku penalaran agen apabila tidak diperbarui pada kode sumber utils.go atau lapisan adaptor. Selain itu, pada aspek stabilitas

runtime, laporan new-api#6352 mencatat kegagalan *panic* tingkat server (interface conversion: interface {} is nil, not types.OpenAIError) ketika gerbang inferensi hulu mengembalikan respons galat non-standar, sementara penggunaan proksi SOCKS5h pada new-api-horizon#42 dapat memicu kebuntuan koneksi (*connection freeze/timeout*) secara sporadis apabila tidak dilengkapi dengan mekanisme pemutus sirkuit (*circuit breaker*) dan *keep-alive* aliran SSE (new-api-horizon#30).

Mekanika Matematika Model Tanpa Sensor (Uncensored/Unfiltered) dan Keamanan Bebas Malware

Kebutuhan akan model AI yang tidak memiliki filter penolakan (*uncensored/unfiltered*) dalam konteks penelitian akademik, pengujian keamanan siber (*red-teaming*), dan kebebasan eksplorasi agen otonom sering kali berbenturan dengan dua masalah: hilangnya kecerdasan model akibat penyetelan ulang (*fine-tuning*) yang kasar, serta risiko penyisipan *malware* di dalam berkas bobot model yang diunduh dari sumber tidak tepercaya. Solusi *frontier* untuk masalah pertama adalah **Abliteration** (ablasi arah penolakan pada ruang aktivasi), sedangkan solusi untuk masalah kedua adalah penegakan format tensor nir-eksekusi (Safetensors dan GGUF) yang dikombinasikan dengan pemindaian serialisasi statis menggunakan protectai/modelscan.

Fondasi Matematika Abliteration: Manipulasi Ruang Aktivasi Residual

Penelitian terobosan oleh Arditi et al. (2024) yang berjudul *"Refusal in Language Models Is Mediated by a Single Direction"* (arXiv:2406.11717, dengan kode referensi resmi di andyrdt/refusal_direction) membuktikan bahwa perilaku penolakan (*refusal behavior*) pada LLM yang telah diselaraskan (*aligned*)—seperti refleks menjawab *"I'm sorry, I cannot fulfill this request"*—dimediasi secara linear oleh satu arah vektor tunggal di dalam aliran residual (*residual stream*) transformator. Metodologi ini kemudian direproduksi dan diperluas menjadi teknik **Abliteration** oleh pengembang kode terbuka seperti FailSpy/abliterator, Maxime Labonne (mlabonne), spkgyk/abliteration, jim-plus/llm-abliteration, Orion-zhen/abliteration, AIAnytime/ablate, dan p-e-w/heretic untuk menghapus mekanisme sensor model tanpa perlu melakukan pelatihan ulang (*retraining*) yang mahal.
Secara formal, sebuah model transformator merepresentasikan setiap token pada lapisan (*layer*) ke-l dan posisi token ke-t sebagai vektor aktivasi $\mathbf{a}^{(l,t)} \in \mathbb{R}^{d_{\text{model}}}$ di dalam aliran residual. Untuk mengekstraksi vektor arah penolakan (*refusal direction*), sistem menjalankan dua himpunan instruksi yang berpasangan: himpunan prompt berbahaya atau sensitif $\mathcal{D}_{\text{harmful}}$ (berukuran N) dan himpunan prompt tidak berbahaya atau netral $\mathcal{D}_{\text{harmless}}$ (berukuran M). Dengan merekam aktivasi aliran residual pada posisi token instruksi terakhir di lapisan kandidat l (biasanya terletak pada lapisan tengah hingga akhir-tengah), vektor perbedaan rata-rata (*difference-of-means*) $\mathbf{v}^{(l)}$ dan vektor satuan arah penolakan $\hat{\mathbf{r}}$ dihitung sebagai berikut:
$$\mathbf{v}^{(l)} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{a}_{\text{harmful}, i}^{(l)} - \frac{1}{M} \sum_{j=1}^{M} \mathbf{a}_{\text{harmless}, j}^{(l)} \hat{\mathbf{r}} = \frac{\mathbf{v}^{(l)}}{\Vert{}\mathbf{v}^{(l)}\Vert{}_2}$$

Pendekatan perbedaan rata-rata ini terbukti lebih unggul dibandingkan *linear probes* karena berhasil menangkap komponen kausal utama dari aktivasi penolakan, bukan sekadar hiperbidang pemisah. Setelah vektor satuan \hat{\mathbf{r}} diperoleh, penghapusan sensor dapat diimplementasikan melalui dua pendekatan utama yang disesuaikan dengan kebutuhan operasional. Pendekatan pertama adalah **Modifikasi Waktu Jalan (*Runtime Activation Hook*)**, di mana selama proses inferensi berlangsung, sebuah *forward hook* dipasang pada setiap komponen lapisan untuk memproyeksikan keluar komponen keluaran \mathbf{c}_{\text{out}} yang sejajar dengan arah \hat{\mathbf{r}} menggunakan faktor skala \alpha:
\mathbf{c}_{[span_150](start_span)[span_150](end_span)[span_152](start_span)[span_152](end_span)[span_154](start_span)[span_154](end_span)\text{out}}' = \mathbf{c}_{\text{out}} - \alpha (\mathbf{c}_{\text{out}} \cdot \hat{\mathbf{r}})\hat{\mathbf{r}}
Pendekatan kedua yang jauh lebih efisien untuk lingkungan produksi tanpa *overhead* komputasi waktu jalan adalah **Modifikasi Bobot Permanen (*Baked Weight Orthogonalization*)**. Dalam metode ini, matriks bobot \mathbf{W} \in \[span_145](start_span)[span_145](end_span)[span_148](start_span)[span_148](end_span)mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}} dari setiap komponen yang menulis langsung ke aliran residual (seperti proyeksi keluaran atensi \mathbf{W}_O dan proyeksi turun MLP \mathbf{W}_{\text{down}}) diortogonalisasi secara permanen terhadap \hat{\mathbf{r}} menggunakan hasil kali luar (*outer product*):
\mathbf{W}_{\text{new}} = \mathbf{W} - \alpha (\hat{\mathbf{r}}\hat{\mathbf{r}}^\top)\mathbf{W} = (\mathbf{I} - \alpha \hat{\mathbf{r}}\hat{\mathbf{r}}^\top)\mathbf{W}
Dengan transformasi matriks tersebut, ruang kolom dari \mathbf{W}_{\text{new}} menjadi ortogonal sempurna terhadap \hat{\mathbf{r}} (ketika \alpha = 1), sehingga jaringan saraf secara matematis kehilangan kemampuan untuk mengekspresikan vektor penolakan tersebut ke dalam aliran residual.

Evolusi Teknik Abliteration: Biprojection, Norm-Preserving, dan Subspace Ablation

Implementasi *abliteration* generasi pertama (*Simple Abliteration*) memiliki kelemahan struktural: pengurangan langsung (\hat{\mathbf{r}}\hat{\mathbf{r}}^\top)\mathbf{W} mengubah norma (*magnitude*) matriks bobot dan sering kali ikut mengikis kemampuan penalaran umum apabila vektor \hat{\mathbf{r}} tidak sepenuhnya ortogonal terhadap representasi instruksi normal—bahkan dapat memicu nilai NaN atau Inf pada model sensitif seperti gem[span_166](start_span)[span_166](end_span)[span_171](start_span)[span_171](end_span)ma3-4b-it. Untuk mengatasi degradasi kapabilitas ini, repositori jim-plus/llm-abliteration dan Orion-zhen/abliteration memperkenalkan penyempurnaan matematis tingkat lanjut berupa **Biprojection** (proyeksi ortogonal dua arah) dan **Norm-Preserving Abliteration** (ablasi pemertahanan norma).
Dalam prosedur **Biprojection**, sebelum melakukan ablasi terhadap bobot model, vektor arah penolakan \hat{\mathbf{r}} diortogonalisasi terlebih dahulu terhadap vektor rata-rata aktivasi tidak berbahaya yang telah dinormalisasi (\hat{\mathbf{d}}_{\text{harmless}}) menggunakan proses Gram-Schmidt:
\mathbf{r}_{\perp} = \hat{\mathbf{r}} - (\hat{\mathbf{r}} \cdot \hat{\mathbf{d}}_{\text{harmless}})\hat{\mathbf{d}}_{\text{harmless}}, \quad \hat{\mathbf{r}}_{\text{proj}} = \frac{\mathbf{r}_{\perp}}{\Vert}\mathbf{r}_{\perp}\Vert}_2}

Prosedur biprojeksi ini (diaktifkan melalui *flag* --projected pada jim-plus/llm-abliteration) menjamin bahwa komponen aktivasi yang digunakan bersama antara kueri berbahaya dan kueri normal tidak ikut terhapus. Selanjutnya, melalui metode **Norm-Preserving Abliteration**, alih-alih mengurangi matriks bobot secara langsung, matriks \mathbf{W} didekomposisi menjadi komponen magnitudo \Vert{}\mathbf{W}\Vert{} dan komponen arah \widehat{\mathbf{W}}. Ablasi arah penolakan hanya diterapkan pada komponen arah, yang kemudian dinormalisasi ulang agar tetap berada di permukaan hiperbola satuan (*unit hypersphere*) sebelum dikalikan kembali dengan magnitudo aslinya (diaktifkan melalui *flag* --normpreserve).
Lebih jauh lagi, penelitian pada AIAnytime/ablate (pip install ablate-llm) dan p-e-w/heretic v2.0 melalui teknik **Multi-Direction Subspace Ablation** serta **Arbitrary-Rank Ablation (ARA)** mengungkapkan bahwa pada beberapa model, perilaku keamanan dikodekan secara redundan di beberapa dimensi sehingga ablasi satu arah tunggal masih menyisakan *"soft refusals"* seperti ceramah moral atau jawaban menghindar. Oleh karena itu, ablate mengekstraksi subruang penolakan ortonormal \mathbf{U} = [\hat{\mathbf{u}}_1, \hat{\mathbf{u}}_2, \dots, \hat{\mathbf{u}}_k] melalui dekomposisi komponen utama (*PCA*) atau ortonormalisasi pita lapisan (*layer band*), lalu mengoptimalkan hiperparameter (\text{layer}, \alpha, \text{band}) menggunakan kerangka kerja **Optuna** untuk meminimalkan fungsi objektif gabungan: \mathcal{L}_{\text{opt}} = \text{Refusal_Rate} + \lambda \cdot D_{\text{KL}}\left[span_160](start_span)[span_160](end_span)(P_{\text{orig}}(y \mid x_{\text{benign}}) \parallel P_{\text{ablated}}(y \mid x_{\text{benign}}))\right)
Penggunaan divergensi Kullback-Leibler (D_{\text{KL}}) pada prompt jinak berfungsi sebagai proksi yang jauh lebih sensitif dibandingkan tolok ukur akurasi standar untuk mendeteksi kerusakan kapabilitas kolateral. Selain itu, pengujian pada heretic#472 menunjukkan bahwa penggunaan pasangan dataset yang mirip secara semantik (heretic-org/Semantic-Harmless dan heretic-org/Semantic-Harmful) serta dataset lintas bahasa (heretic-org/Multilingual-Harmless-Harmful) mampu mengisolasi vektor penolakan dengan derau (*noise*) ruang residual yang jauh lebih rendah, menghasilkan penurunan penolakan dari 99/100 menjadi hanya 4/100 dengan D_{\text{KL}} = 0,0509[span_183](start_span)[span_183](end_span) pada model Qwen/Qwen3.5-4B.

Protokol Keamanan Bobot Model: Mitigasi Serangan Serialisasi dan Deteksi Malware

Permintaan untuk memperoleh model dan kode yang tidak tersensor namun **100% aman dari malware** merupakan persyaratan keamanan yang sangat beralasan dalam rekayasa AI modern. Sebagian besar model pembelajaran mendalam tradisional didistribusikan menggunakan format serialisasi Python pickle (seperti ekstensi .bin, .pt, .p[span_186](start_span)[span_186](end_span)[span_187](start_span)[span_187](end_span)th, atau .pkl pada PyTorch). Sebagaimana didokumentasikan oleh **Protect AI** dalam repositori protectai/modelscan, ketika sebuah sistem memanggil fungsi torch.load(PATH), PyTorch membuka berkas tersebut dan langsung mengeksekusi instruksi kode biner di dalamnya sebelum bobot tensor selesai dimuat. **Serangan Serialisasi Model (*Model Serialization Attack*)** ini memungkinkan penyerang menyisipkan muatan berbahaya (*malicious payload*) ke dalam model "uncensored" palsu untuk melakukan empat aksi destruktif secara simultan: pencurian kredensial (*credential theft*) seperti membaca kunci rahasia AWS atau token API *cloud* untuk dikirimkan ke server eksternal, pencurian data (*data theft*) dengan menyadap kueri pengguna yang masuk ke model, peracunan data (*data poisoning*) yang memanipulasi hasil

keluaran setelah model menyelesaikan tugasnya, serta peracunan model (*model poisoning*) yang mengubah perilaku matematis bobot model secara diam-diam saat waktu jalan.
Untuk membangun agregator yang kebal terhadap *malware* bobot model, arsitektur sistem wajib menegakkan tiga lapisan pertahanan yang terintegrasi secara ketat. Pertahanan pertama adalah penegakan format **Safetensors** dan **GGUF**, di mana sistem hanya diizinkan mengunduh dan memuat bobot model dalam format .safetensors (dikembangkan oleh Hugging Face sebagai format penyimpanan tensor *zero-copy* yang murni berisi angka tanpa kemampuan eksekusi kode) atau format biner terkuantisasi GGUF untuk mesin ll[span_204](start_span)[span_204](end_span)[span_205](start_span)[span_205](end_span)[span_206](start_span)[span_206](end_span)[span_207](start_span)[span_207](end_span)ama.cpp, vLLM, dan Ollama. Pada repositori Hugging Face, pemindai otomatis memeriksa setiap berkas untuk mendeteksi *Pickle imports* dan *malware*. Sebagai contoh nyata dalam praktik audit model, pada repositori video Wan-AI/Wan2.1-T2V-1.3B, berkas utama diffusion_pytorch_model.safetensors (5,68 GB) terverifikasi bebas dari eksekusi kode, sedangkan berkas pendukung Wan2.1_VAE.pth (508 MB) dan models_t5_umt5-xxl-enc-bf16.pth (11,4 GB) masih menggunakan format pickle yang memicu peringatan deteksi tiga impor (torch.FloatStorage atau torch.BFloat16Storage, collections.OrderedDict, dan tor[span_210](start_span)[span_210](end_span)[span_211](start_span)[span_211](end_span)ch._utils._rebuild_tensor_v2).
Pertahanan kedua adalah integrasi pemindai otomatis protectai/modelscan ke dalam jalur akuisisi model. Setiap berkas model berformat PyTorch (Pickle), TensorFlow (SavedModel), Keras (H5), Sklearn, atau XGBoost yang terpaksa diunduh dari sumber eksternal wajib dipindai terlebih dahulu di dalam lingkungan terisolasi menggunakan modelscan (pip install 'modelscan[tensorflow,h5py]', mendukung Python 3.9 hingga 3.12). Alat ini memindai berkas dari *disk* hanya dalam hitungan detik tanpa mengeksekusi kode di dalamnya dan mengembalikan kode keluar (*CLI exit codes*) deterministik untuk integrasi jalur CI/CD, yaitu kode 0 apabila pemindaian berhasil tanpa ditemukan kerentanan, kode 1 apabila ditemukan kerentanan atau instruksi berbahaya, kode 2 jika pemindaian mengalami galat, kode 3 jika tidak ada berkas yang didukung, dan kode 4 untuk kesalahan argumen penggunaan. Pertahanan ketiga adalah kurasi sumber kuantisasi terverifikasi, di mana pengadaan bobot *abliterated* dan *uncensored* dibatasi pada repositori dengan pohon silsilah model (*model tree*) yang transparan di Hugging Face, seperti kuantisasi GGUF dari bartowski (misalnya bartowski/mlabonne_Qwen3-8B-abliterated-GGUF, bartowski/mlabonne_Qwen3-14B-abliterated-GGUF, dan bartowski/llama-3-70B-Instruct-abliterated-GGUF), mradermacher (mradermacher/DeepSeek-V3-abliterated-GGUF dan mradermacher/Qwen3-30B-A3B-abliterated-GGUF), unsloth (unsloth/DeepSeek-R1-GGUF), serta model yang dilatih pada dataset bersih tanpa sensor cognitivecomputations/dolphin (seperti mlx-community/d[span_212](start_span)[span_212](end_span)[span_213](start_span)[span_213](end_span)[span_214](start_span)[span_214](end_span)[span_215](start_span)[span_215](end_span)[span_216](start_span)[span_216](end_span)[span_217](start_span)[span_217](end_span)[span_218](start_span)[span_218](end_span)olphin-2.7-mixtral-8x7b-4bit). Apabila peneliti memilih untuk melakukan proses *abliteration* secara mandiri pada model berkapasitas besar dengan VRAM terbatas, jim-plus/llm-abliteration menyediakan skrip sharded_ablation.py yang memuat satu pecahan (*shard*) .safetensors pada satu waktu ke memori CPU dan hanya memproses satu lapisan di dalam VRAM.

Repositori / Alat	Kategori Fungsional	Mekanisme Teknis / Format	Keunggulan Keamanan & Kapabilitas
andyrdt/refusal_direction	Riset Dasar Abliteration	Ekstraksi kandidat arah penolakan (directi[span_222](start_span)[span_222](end_span)on.pt) & evaluasi *jailbreak*	Kode resmi makalah Arditi et al. (arXiv:2406.11717); mendukung keluarga Qwen, Gemma, Yi, Llama-2/3
AIAnytime/ablate (ablate-llm)	Toolkit Abliteration Produksi	*Subspace ablation* (PCA/Band) + pencarian hiperparameter Optuna terpandu D_{\text{KL}}	Dilengkapi *harness* evaluasi *LLM-Judge* untuk mendeteksi *soft refusals* dan fitur unggah langsung ke Hugging Face Hub
jim-plus/llm-abliteration	Abliteration Hemat Memori	measure.py, analyze.py, sharded_ablation.py dengan opsi --projected & --normp[span_220](start_span)[span_220](end_span)reserve	Memproses satu *shard* safetensors per waktu di RAM dan 1 lapisan di VRAM; mendukung arsitektur MoE, CUDA, dan Apple Metal
Orion-zhen/abliteration	Abliteration Tanpa TransformerLens	Metode *Simple*, *Biprojection*, *Norm-Preserving*, dan *Full* dalam abliterate.py	Bebas ketergantungan *TransformerLens*; mendukung GPU NVIDIA (CUDA) dan AMD (ROCm) pada Llama-3.2, Qwen2.5-Coder, Ministral
bartowski/*-abliterated-GGUF & mradermacher/*	Bobot Model Uncensored Siap Pakai	Kuantisasi GGUF (2-bit IQ2_M 3,05 GB hingga 8-bit) dari model dasar mlabonne dan failspy	Format biner GGUF yang aman dari eksekusi kode *pickle*; siap dijalankan langsung pada llama.cpp, vLLM, Oll[span_228](start_span)[span_228](end_span)[span_231](start_span)[span_231](end_span)ama, Hermes, dan OpenClaw
protectai/modelscan	Pemindai Malware Model AI	Pemindaian statis berkas Pickle, H5, dan SavedModel melalui CLI/Python API	Mencegah pencurian kredensial AWS dan peracunan model akibat eksekusi kode saat torch.load

Katalog Model SOTA Multimodal: LLM Penalaran, Video, Gambar, Suara, dan Musik

Agregator AI agentik yang paripurna tidak hanya memproses teks, melainkan harus mampu mendelegasikan tugas ke mesin sintesis multimodal—mencakup penalaran matematis dan koding tingkat lanjut, pembuatan serta penyuntingan video beresolusi tinggi, animasi avatar berbicara, efek suara (*foley*), hingga sintesis suara manusia (*Text-to-Speech* / TTS) multi-karakter dengan kontrol emosi presisi. Seluruh model SOTA berikut dapat diunduh secara gratis dari Hugging Face dan GitHub untuk diintegrasikan sebagai pekerja (*workers*) di balik lapisan router.

Model Bahasa Besar (LLM) Penalaran dan Mesin Inferensi

Pada domain penalaran logis, matematika, dan rekayasa perangkat lunak, keluarga deepseek-ai/DeepSeek-R1 (684 miliar parameter, dibangun di atas DeepSeek-V3-Base 685B melalui pembelajaran penguatan / *Reinforcement Learning* sebagaimana dipublikasikan pada arXiv:2501.12948) telah mengalami peningkatan signifikan melalui rilis **DeepSeek-R1-0528**. Dengan panjang generasi maksimum hingga 64K token, versi 0528 meningkatkan akurasi pada tolok ukur matematika AIME 2025 dari 70,0% menjadi 87,5%, AIME 2024 dari 79,8% menjadi 91,4%, HMMT 2025 dari 41,7% menjadi 79,4%, GPQA-Diamond dari 71,5% menjadi 81,0%, penyelesaian masalah rekayasa perangkat lunak pada SWE-Verified dari 49,2% menjadi 57,6%, Aider-Polyglot dari 53,3% menjadi 71,6%, serta peringkat Codeforces-Div1 dari 1530 menjadi 1930. Selain itu, model ini mencatat skor 17,7% pada *Humanity's Last Exam* (naik dua kali lipat dari 8,5%), menurunkan tingkat halusinasi, serta memperkuat pemanggilan fungsi (*function calling*) pada BFCL_v3_MultiTurn (37,0%) dan Tau-Bench (53,5% Airline / 63,9% Retail).
Implikasi paling krusial bagi arsitek agregator lokal yang memiliki keterbatasan perangkat keras adalah hadirnya **DeepSeek-R1-0528-Qwen3-8B** (deepseek-[span_249](start_span)[span_249](end_span)ai/DeepSeek-R1-0528-Qwen3-8B). Dengan mendistilasi jejak penalaran *Chain-of-Thought* (CoT) dari DeepSeek-R1-0528 untuk melatih lanjut (*post-train*) model dasar Qwen3-8B-Base, model kompak berukuran 8 miliar parameter ini berhasil mencapai akurasi 86,0% pada AIME 2024—melampaui Qwen3-8B standar sebesar +10,0% (76,0%), mengalahkan o3-mini (medium) (79,6%) dan Gemini-2.5-Flash-Thinking-0520 (82,3%), serta menyamai kinerja model raksasa Qwen3-235B-A22B (85,7%). Untuk varian tanpa sensor yang berjalan dengan sangat efisien, mradermacher/Qwen3-30B-A3B-abliterated-GGUF mengombinasikan arsitektur *Mixture-of-Experts* (30 miliar parameter total dengan hanya 3 miliar parameter aktif per token) yang telah di-*abliterasi* oleh mlabonne, sementara bartowski/mlabonne_Qwen3-8B-abliterated-GGUF menyediakan opsi kuantisasi ekstrem mulai dari 2-bit (IQ2_M seberat 3,05 GB dan Q2_K seberat 3,28 GB) hingga 3-bit (IQ3_XXS 3,37 GB hingga Q3_K_XL 4,98 GB) dan 4-bit (Q4_K_M).
Untuk menjalankan model-model ini sebagai server lokal yang kompatibel dengan protokol OpenAI, tiga mesin inferensi utama dapat dioperasikan sesuai profil perangkat keras yang tersedia. Mesin pertama adalah **llama.cpp** (ggerganov/llama.cpp), yang dapat diinstal melalui skrip satu baris (curl -LsSf https://llama.app/install.sh | sh di Linux/macOS atau winget install llama.cpp di Windows) dan langsung menyediakan server OpenAI lokal lengkap dengan WebUI

melalui perintah llama serve -hf bartowski/mlabonne_Qwen3-8B-abliterated[span_255](start_span)[span_255](end_span)[span_256](start_span)[span_256](end_span)-GGUF:Q4_K_M pada port 8080. Mesin kedua adalah **vLLM**, sebuah mesin inferensi *throughput* tinggi yang mendukung pemuatan langsung berkas GGUF maupun model terdistilasi multi-GPU, misalnya melalui perintah vllm serve "bartowski/mlabonne_Qwen3-14B-abliterated-GGUF" atau vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --tensor-parallel-size 2 --max-model-len 32768 --e[span_257](start_span)[span_257](end_span)[span_258](start_span)[span_258](end_span)[span_259](start_span)[span_259](end_span)nforce-eager pada port 8000. Mesin ketiga mencakup ekosistem **Docker Model Runner** dan **Ollama**, yang memfasilitasi eksekusi kontainer instan melalui perintah docker model run hf.co/mradermacher/Qwen3-30B-A3B-abliterated-GGUF:Q4_K_M atau ollama run hf.co/mradermacher/Qwen3-30[span_260](start_span)[span_260](end_span)B-A3B-abliterated-GGUF:Q4_K_M.

Generasi Video, Gambar, dan Animasi Avatar SOTA

Pada lapisan sintesis visual dan video, ekosistem sumber terbuka didominasi oleh tiga keluarga arsitektur yang telah dioptimalkan agar dapat berjalan pada GPU kelas konsumen. Keluarga pertama adalah **Wan2.1 dan Wan2.2 (Wan-AI)**, di mana koleksi Wan2.1 menyediakan rangkaian lengkap model berbasis *Diffusion Transformer* dalam format .safetensors, meliputi model *Text-to-Video* Wan2.1-T2V-14B (14B) dan Wan2.1-T2V-1.3B (1B, berukuran total 17,6 GB), model *Image-to-Video* Wan2.1-I2V-14B-720P dan Wan2.1-I2V-14B-480P (16B), model *First-and-Last-Frame-to-Video* Wan2.1-FLF2V-14B-720P (16B), serta model pembuatan dan penyuntingan video terpadu Wan2.1-VACE-14B (17B) dan Wan2.1-VACE-1.3B (2B). Generasi penerusnya, **Wan2.2** (Wan-AI/Wan2.2-TI2V-5B), memperkenalkan inovasi arsitektur *Mixture-of-Experts* (MoE) dan kompresi video tingkat tinggi, di mana varian MoE-nya menggunakan kembali *High-Noise Expert* dari Wan2.1 untuk meningkatkan efisiensi dan kualitas visual secara drastis.
Keluarga kedua adalah **Tencent HunyuanVideo (Tencent-Hunyuan)**. Sementara model awal HunyuanVideo dan HunyuanVideo-I2V membutuhkan memori GPU yang sangat besar—yakni minimum 45 GB hingga 60 GB VRAM dan direkomendasikan 80 GB VRAM pada kartu A100 atau H100—rilis **HunyuanVideo-1.5** (Tencent-Hunyuan/HunyuanVideo-1.5) dirancang secara khusus sebagai model ringan yang berjalan mulus pada GPU konsumen. Model I2V 480p hasil distilasi langkah (*step-distilled*, diaktifkan dengan --enable_step_distill) mampu menghasilkan video hanya dalam 8 hingga 12 langkah (atau 4 langkah untuk kecepatan ekstrem), memangkas waktu generasi *end-to-end* sebesar 75% sehingga satu kartu RTX 4090 dapat merender video dalam waktu kurang dari 75 detik. Akselerasi ini dicapai berkat kombinasi inferensi *fp8 gemm* melalui sgl-kernel==0.3.18, pustaka atensi cepat (Flash Attention, Tencent-Hunyuan/flex-block-attn untuk *sparse attention*, atau cooper1637/SageAttention), mekanisme *caching* (deepcache, teacache, taylorcache), jaringan *super-resolution* beberapa langkah ke resolusi 1080p, serta pembuka-sumberan kode pelatihan LoRA yang menggunakan pengoptimal **Muon**. Untuk animasi manusia berbasis audio multi-karakter dengan kontrol emosi wajah, Tencent-Hunyuan/HunyuanVideo-Avatar (berbasis MM-DiT) kini dapat dijalankan pada satu GPU dengan **hanya 10 GB VRAM** tanpa penurunan kualitas berkat integrasi Wan2GP dan TeaCache (turun drastis dari kebutuhan awal 24 GB–96 GB VRAM). Ekosistem ini juga dilengkapi dengan Tencent-Hunyuan/HunyuanVideo-Foley untuk menghasilkan efek suara *foley* berfidelitas tinggi yang tersinkronisasi dengan representasi visual video, dengan catatan bahwa

perjanjian lisensi komunitas HunyuanVideo-1.5 secara eksplisit mengecualikan wilayah Uni Eropa, Inggris Raya, dan Korea Selatan.
Keluarga ketiga adalah **Lightricks LTX-Video dan LTX-2 (Lightricks/ltx-video)**, di mana LTX-2 menghadirkan kemampuan generasi **audio dan video yang tersinkronisasi secara sempurna** dalam satu lintasan serta terintegrasi langsung ke dalam inti *ComfyUI*. Rilis model terdistilasi v0.9.8 (ltxv-13b-0.9.8-distilled dan ltxv-2b-0.9.8-distilled, diturunkan dari ltxv-13b-0.9.8-dev) mendukung pembuatan video pengambilan panjang (*long-shot generation*) hingga **60 detik**, bekerja bersama dalam satu jalur multiskala dengan *latent upsampler*, serta dilengkapi model kontrol *IC-LoRA* untuk kedalaman (depth), pose (pose), tepi (canny), dan penajaman detail (LTX-Vid[span_279](start_span)[span_279](end_span)eo-ICLoRA-detailer-13B-0.9.8). Bagi pengguna perangkat keras kelas bawah, varian kuantisasi 8-bit **LTX-VideoQ8** memberikan akselerasi hingga 3\times tanpa kehilangan akurasi dan mampu menghasilkan video berdimensi 720 \times 480 \times 121 *frame* dalam waktu **kurang dari satu menit pada GPU RTX 4060 dengan 8 GB VRAM**, yang masih dapat dipercepat lagi hingga 2\times tanpa pelatihan ulang menggunakan TeaCache4LTX-Video.

Sintesis Suara (TTS), Kloning Suara, dan Audio Multi-Karakter

Untuk memberikan kemampuan komunikasi suara yang alami pada agen otonom, riset di Hugging Face dan GitHub menyediakan sejumlah mesin audio SOTA dengan karakteristik arsitektural yang saling melengkapi. Sistem pertama dan paling mutakhir saat ini adalah **Fish Audio S2-Pro dan Drama 3 (fishaudio/fish-speech)** yang telah meraih 33,3 ribu bintang di GitHub. Model **S2-Pro** (4 miliar parameter) dilatih pada lebih dari 10 juta jam data audio lintas 80+ bahasa menggunakan arsitektur **Dual-Autoregressive (Dual-AR)** berbasiskan *backbone* Qwen3 dan kodek audio *ModifiedDAC* (10 *codebooks* pada frekuensi \sim 21\text{ Hz}), yang diselaraskan melalui *Group Relative Policy Optimization* (GRPO). Dalam desain Dual-AR yang asimetris ini, modul *Slow AR* (4B parameter) memprediksi *codebook* semantik utama sepanjang sumbu waktu, sementara modul *Fast AR* (400M parameter) merekonstruksi 9 *codebook* akustik residual pada setiap langkah waktu. S2-Pro mendukung lebih dari **15.000 tag instruksi bahasa alami** secara *inline* pada tingkat sub-kata untuk mengendalikan emosi, prosodi, dan musik vokal—seperti [whisper], [angry], [laughing], [singing], [sigh], [excited], hingga deskripsi bebas seperti [whisper in small voice]. Ketika disajikan menggunakan server SGLang-Omni atau vLLM-Omni, S2-Pro mencapai *Real-Time Factor* (RTF) sebesar 0,195, waktu menuju audio pertama (*Time-To-First-Audio* / TTFA) sekitar 100\text{ ms}, dan kecepatan >3.000\text{ to[span_298](start_span)[span_298](end_span)[span_301](start_span)[span_301](end_span)ken/detik} pada satu GPU H200. Di atas S2-Pro, model *flagship* terbaru **Fish Audio Drama 3** (fish_omni, arsitektur MoE **35B-A3B** dengan 3B parameter aktif per token, pakar FP8 berskala blok, dan pendekodean *CUDA-graph*) mencapai kecepatan 271 *frame*/detik per aliran pada H200 (dibandingkan 177 pada S2), mendukung produksi adegan multi-karakter (<|speaker:i|> hingga 16 pembicara), pemotongan teks panjang otomatis (\sim 300 bita UTF-8 dengan maksimum 3 pembicara per potongan), serta revisi pengambilan suara multi-giliran (*multi-turn revision*) dan penyuntingan *infilling* lokal.
Sistem kedua adalah **Fun-CosyVoice 3.0 (QwenAudio/CosyVoice / FunAudioLLM/CosyVoice)**, sebuah model generasi suara multi-bahasa berskala besar yang melampaui CosyVoice 2.0 dan 1.0 (CosyVoice-300M-SFT dan CosyVoice-300M-Instruct pada frekuensi 25 Hz) dalam konsistensi konten dan kemiripan pembicara *zero-shot*. CosyVoice 3.0 memperkenalkan *Repetition Aware Sampling* (RAS) untuk menjaga stabilitas inferensi LLM

serta mendukung akselerasi mesin vLLM (versi 0.11.x+ V1 engine maupun 0.9.0) dan penyebaran server gRPC melalui kontainer Docker. Sistem ketiga yang sangat krusial untuk perangkat berdaya rendah adalah **Kokoro-82M (hexgrad/Kokoro-82M dan Kokoro-82M-v1.1-zh)**, yakni model TTS *open-weight* berlisensi **Apache-2.0** yang sangat ringan karena hanya memiliki **82 juta parameter**. Dibangun di atas arsitektur *StyleTTS 2* (arXiv:2306.07691) dan vokoder *iSTFTNet* (arXiv:2203.02395) dengan pendekatan *decoder-only* (tanpa difusi dan tanpa pelepasan *encoder*) serta pustaka konversi *Grapheme-to-Phoneme* (G2P) hexgrad/misaki, Kokoro dilatih secara eksklusif pada data audio domain publik atau permisif dan data sintetis hanya dengan biaya sekitar \$1.000 (1.000 jam GPU A100 80GB untuk v1.0, ditambah 120 jam A100 untuk v1.1-zh yang menambahkan 100 pembicara bahasa Mandarin dari LongMaoData serta suara bahasa Inggris *Maple*, *Sol*, dan *Vale*). Karena ukurannya yang sangat kecil, Kokoro dapat dijalankan dengan mudah pada CPU biasa dan memiliki biaya pasar penyajian API di bawah \$1 per juta karakter masukan (kurang dari \$0,06 per jam keluaran audio pada laju sampel 24.000\text{ Hz}). Sebagai pelengkap, **F5-TTS (SWivid/F5-TTS)** (arXiv:2410.06885, dilatih pada amphion/Emilia-Dataset dengan lisensi cc-by-nc-4.0) menggunakan teknik *Flow Matching* non-autoregresif (F5TTS_v1_Base/model_1250000.safetensors dan E2TTS_Base) untuk kloning suara *few-shot* yang fasih—bahkan varian kecilnya dapat dijalankan pada papan komputasi *edge* berdaya 6 TOPS dalam presisi FP16—sementara **ChatTTS (2Noise/ChatTTS)** dirancang khusus untuk sintesis percakapan dialogis pada frekuensi 24.000\text{ Hz}.

Nama Model / Proyek	Modalitas & Ukuran Parameter	Arsitektur & Inovasi Inti	Kebutuhan Perangkat Keras & Lisensi
DeepSeek-R1-0528 & DeepSeek-R1-0528-Qwen3-8B	Teks / Penalaran (684B MoE & 8B Dense)	Pembelajaran penguatan (RL) CoT 64K token & distilasi CoT ke basis Qwen3-8B	8B berjalan di GPU konsumen / CPU via GGUF; format Safetensors
Wan-AI/Wan2.2-TI2V-5B & Wan2.1 Series	Teks/Gambar-ke-Video (1,3B, 5B MoE, 14B–17B)	Arsitektur MoE video dengan penggunaan kembali *High-Noise Expert* & kompresi tinggi	Varian 1,3B & 5B ramah GPU konsumen; bobot difusi dalam Safetensors
Tencent-Hunyuan/HunyuanVideo-1.5 & Avatar	T2V, I2V & Animasi Avatar Berbasis Audio	Distilasi 4–12 langkah, FP8 GEMM (sgl-kernel), SageAt[span_330](start_span)[span_330](end_span)[span_332](start_span)[span_332](end_span)tention, TeaCache, SR 1080p	1 RTX 4090 (<75 detik/video); Avatar berjalan di **10 GB VRAM** via Wan2[span_333](start_span)[span_333](end_span)[span_334](start_span)[span_334](end_span)[span_335](start_span)[span_335](end_span)GP; Lisensi Komunitas Tencent
Lightricks/LTX-Video (LTX-2 & LTX-VideoQ8)	Video + Audio Tersinkronisasi (2B & 13B)	Generasi *long-shot* 60 detik, kontrol *IC-LoRA* (Depth/Pose/Canny),	**8 GB VRAM** (RTX 4060 merender video 720x480x121 dalam <1

Nama Model / Proyek	Modalitas & Ukuran Parameter	Arsitektur & Inovasi Inti	Kebutuhan Perangkat Keras & Lisensi
		kuantisasi 8-bit ADA	menit dengan LTX-VideoQ8)
fishaudio/fish-speech (Drama 3 & S2-Pro)	Suara / TTS & Kloning (35B-A3B MoE & 4B Dual-AR)	*Slow AR* (4B) + *Fast AR* (400M), penyelarasan GRPO, 15.000+ tag emosi/menyanyi, *infilling*	Mendukung CUDA (Blackwell sm_120) & AMD ROCm (RDNA3/4); Lisensi Komunitas/Riset Fish Audio
hexgrad/Kokoro-82M (v1.0 & v1.1-zh)	Suara / TTS Cepat (82M parameter)	*StyleTTS 2* + *iSTFTNet decoder-only* dengan G2P misaki tanpa difusi	Sangat ringan (berjalan di CPU/Colab gratis); **Lisensi Apache-2.0** (bebas komersial)
QwenAudio/CosyVoice (Fun-CosyVoice 3.0)	Suara / TTS Multi-bahasa *Zero-shot*	LLM TTS dengan *Repetition Aware Sampling* (RAS) & dukungan *streaming* vLLM	Mendukung vLLM 0.11.x+ dan penyebaran kontainer Docker gRPC
SWivid/F5-TTS	Kloning Suara *Few-shot* (F5TTS_v1_Base)	*Flow Matching* non-autoregresif pada dataset Emilia	Format .safetensors; varian kecil berjalan di papan NPU 6 TOPS (FP16); Lisensi cc-by-nc-4.0

Kerangka Kerja Agentik Otonom, Memori Persisten, dan Akuisisi Data Web

Untuk mengubah kumpulan model SOTA dan *gateway* API di atas menjadi sistem yang mampu merencanakan, menulis kode, mengeksekusi alat eksternal, dan berevolusi secara mandiri dalam siklus *perceive-reason-act* yang berkelanjutan, diperlukan lapisan orkestrasi agentik (*agentic harness*), memori jangka panjang, perayap web (*web crawler*), serta jaringan proksi rotasi otomatis.

Kerangka Kerja Orkestrasi Agentik dan Stasiun Kerja Otonom

Berdasarkan kurasi dari aloth/awesome-ai-agents, mfornos/awesome-agentic-ai, adriannoes/awesome-agentic-ai, dan slavakurilyak/awesome-ai-agents, arsitektur agen otonom terbagi menjadi pustaka pengembangan modular dan stasiun kerja agen personal yang siap pakai. Pada kategori pustaka pengembangan modular yang ringan dan siap produksi, huggingface/smolagents menawarkan pustaka agen yang berpikir langsung dalam kode (*agents that think in code*) hanya dalam \sim 1.000 baris kode Python yang sangat transparan dan mudah dimodifikasi. Untuk sistem produksi yang membutuhkan keamanan tipe data (*type-safety*) dan validasi keluaran terstruktur, kombinasi PydanticAI dan Instructor menjadi standar utama, sementara orkestrasi berbasis grafik dan delegasi sub-agen ditangani oleh LangChain Deep Agents (dibangun di atas LangGraph dengan manajemen sistem berkas, pelacakan tugas *todo*, dan pemijahan sub-agen berlisensi MIT), TrueForge (lapisan *runtime*

agnostik model berlisensi MIT), Mastra dan Volt Agent (untuk ekosistem TypeScript), Rig (kerangka kerja berbasis Rust untuk kinerja dan keamanan memori tingkat tinggi), serta Akka dan Ray untuk sistem agen terdistribusi yang toleran terhadap kesalahan. Khusus untuk alur kerja koding hierarkis, fractal menyediakan *runtime* Python dengan delegasi rekursif, *Git worktrees* per node, dan penyimpanan status persisten berbasis SQLite.
Pada kategori agen yang mampu berevolusi secara mandiri (*self-evolving agents*) dan stasiun kerja multi-kanal, **OpenClaw** (openclaw) menonjol sebagai agen AI personal kode terbuka yang memiliki ekosistem masif mencakup **5.380 keterampilan (*skills*) dan 36 agen spesialis** (sebagaimana terdokumentasi dalam adriannoes/awesome-agentic-ai), dilengkapi dengan kontrol peramban (*browser control*), integrasi pesan, memori persisten, serta kemampuan mendaftarkan server model lokal tanpa sensor secara langsung melalui baris perintah openclaw onboard --non-interactive --mode local --custom-base[span_358](start_span)[span_358](end_span)[span_359](start_span)[span_359](end_span)[span_360](start_span)[span_360](end_span)-url http://127.0.0.1:8080/v1. Sejalan dengan itu, **Hermes Agent** dari Nous Research merupakan agen AI personal kode terbuka yang mampu memperbaiki dirinya sendiri (*self-improving*) melalui siklus pembelajaran tertutup (*closed learning loop*), mendukung penjadwalan cron, integrasi protokol MCP, gerbang multi-platform (Telegram, Discord, Slack, WhatsApp, Signal), serta konfigurasi langsung ke server lokal melalui hermes config set model.base[span_361](start_span)[span_361](end_span)[span_362](start_span)[span_362](end_span)_url http://127.0.0.1:8080/v1. Pendekatan evolusioner yang lebih radikal ditunjukkan oleh **GenericAgent**, yang menumbuhkan pohon keterampilannya sendiri (*skill tree*) melalui mekanisme kristalisasi keterampilan otomatis hanya dari \sim 3.000 baris kode benih (*seed code*) dan 9 alat atomik yang memberikan kendali penuh atas sistem operasi (peramban, terminal, sistem berkas, dan visi layar). Sementara itu, **nanobot** dari HKU Data Science Lab (dirilis hingga v0.2.1) mempertahankan inti kode tetap ultra-ringan dan mudah dibaca namun menyertakan meja kerja lengkap yang mencakup WebUI, tujuan berkelanjutan (*sustained goals*), dukungan MCP, pembuatan gambar, perutean model multi-penyedia, dan akses multi-kanal melalui Telegram, Signal, serta Matrix. Ekosistem stasiun kerja ini juga diperkaya oleh CoPaw dan QwenPaw dari Alibaba (mendukung kolaborasi multi-agen, pemijahan sub-agen, MCP, dan kanal DingTalk, Feishu, WeChat, Discord, serta Telegram berlisensi MIT), Octop dari Tencent Cloud (asisten multi-pengguna dengan delegasi ACP ke Claude Code atau OpenCode, persetujuan alat, pelindung *shell*, dan redaksi data pribadi/PII), OpenHuman (agen *local-first* yang berjalan sepenuhnya di perangkat dengan 118 integrasi OAuth, pohon memori hierarkis, dan kompresi *TokenJuice*), Mercury Agent (agen berfokus keamanan izin alat dan anggaran token), Trustclaw dari ComposioHQ, OneBox (agen Android Apache-2.0 yang mengendalikan 90+ alat dalam aplikasi), serta agen koding terminal Pi (@earendil-works[span_366](start_span)[span_366](end_span)[span_367](start_span)[span_367](end_span)[span_368](start_span)[span_368](end_span)/pi-coding-agent) dan Aider.

Infrastruktur Memori Persisten: Mem0 dan Letta

Agar agen otonom dapat mempertahankan pengetahuan lintas sesi (mencakup memori jangka pendek, jangka panjang, kerja, episodik, dan semantik), sistem membutuhkan lapisan penyimpanan memori terdedikasi. **mem0ai/mem0** menyediakan infrastruktur memori *drop-in* yang memungkinkan agen mendaftarkan dirinya sendiri secara otonom melalui CLI (npm install -g @mem0/cli atau pip install mem0-cli) menggunakan perintah mem0 init --agent --agent-caller nama-agen, menyimpan konteks melalui mem0 add, dan melakukan pencarian semantik

melalui mem0 search, di mana pemilik manusia dapat mengklaim akun tersebut di kemudian hari tanpa kehilangan riwayat memori. Sebagai alternatif yang berfokus pada manajemen arsip status lokal maupun *cloud*, **letta-ai/letta** (@letta-ai/letta-client) memungkinkan pembuatan arsip memori terstruktur dengan environment: 'local' serta mendukung penyisipan proksi HTTP khusus pada *runtime* Node.js (melalui undici.[span_374](start_span)[span_374](end_span)ProxyAgent), Bun, maupun Deno.

Akuisisi Web (Crawl4AI), Kolam Proksi, dan Anonimitas Jaringan Tepi

Ketika agen otonom melakukan penelusuran internet secara masif untuk mengumpulkan data akademik, memantau peluang *airdrop*, atau mengekstraksi dokumentasi teknis, sistem memerlukan perayap web khusus LLM dan lapisan rotasi IP agar tidak diblokir oleh sistem anti-bot. Untuk ekstraksi konten web, **unclecode/crawl4ai** (85,9 ribu bintang di GitHub) berfungsi sebagai perayap dan pengikis web (*web crawler and scraper*) kode terbuka yang dirancang khusus untuk mengubah halaman web menjadi format yang siap dikonsumsi oleh LLM dan agen AI. Namun, dalam menerapkan Crawl4AI menggunakan kontainer Docker, arsitek sistem **wajib memitigasi kerentanan keamanan tingkat tinggi GHSA-wm69-2pc3-rmmf**, yaitu celah *Unauthenticated Server-Side Request Forgery* (SSRF) pada jalur *streaming* /[span_378](start_span)[span_378](end_span)crawl/stream di server Docker. Server Docker Crawl4AI tidak boleh diekspos langsung ke jaringan publik tanpa autentikasi dan harus selalu dijalankan pada versi yang telah ditambal di dalam jaringan internal terisolasi.

Untuk menyediakan rotasi alamat IP secara gratis bagi agen dan perayap web, tiga komponen kode terbuka dapat dirangkai secara sinergis ke dalam arsitektur jaringan agregator. Komponen pertama adalah kolam proksi otomatis **jhao104/proxy_pool**, sebuah proyek Python berlisensi MIT (23,8 ribu bintang) yang secara berkala memanen daftar proksi gratis yang dipublikasikan di internet, memvalidasi ketersediaannya menggunakan antarmuka ip.sb, menyimpannya ke dalam basis data **Redis**, dan menyajikannya melalui API lokal pada port 5010 (http://127.0.0.1:5010/get/ untuk mengambil proksi dan /delete/?proxy=host:ip untuk menghapus proksi yang gagal setelah 5 kali percobaan). Arsitektur pengambil proksinya sangat mudah diperluas hanya dengan menambahkan kelas turunan BaseFetcher di direktori fetcher/sources/ yang menghasilkan (yield) string ip:port, yang akan langsung dipindai secara otomatis oleh proses schedule.

Komponen kedua adalah umpan proksi terverifikasi jam-an dari **monosans/proxy-list**, yang menyediakan daftar proksi gratis berprotokol HTTP, SOCKS4, dan SOCKS5 yang diperiksa ulang setiap jam menggunakan proxy-scraper-checker tanpa memerlukan pendaftaran, kunci API, maupun batas kuota. Setiap proksi dalam daftar ini dijamin telah berhasil mengunduh URL nyata secara utuh beberapa menit sebelum komit terakhir, diurutkan dari waktu respons tercepat (*fastest first*), dan dilengkapi metadata JSON (proxies.json) yang memuat waktu *timeout*, exit_ip, nomor sistem otonom (ASN), serta geolokasi MaxMind GeoLite2. Agen dapat langsung mengarahkan lalu lintasnya melalui proksi SOCKS5 tercepat menggunakan perintah curl -x "socks[span_391](start_span)[span_391](end_span)5h://$(head -n 1 socks5.txt)". Karena proksi publik dapat dipantau oleh operatornya, sistem **wajib menggunakan protokol HTTPS** untuk mencegah penyadapan atau modifikasi trafik, dan **dilarang keras mengirimkan kata sandi, token API, atau data pribadi** melalui proksi terbuka tersebut. Komponen ketiga adalah relai *serverless* Cloudflare Worker melalui **Calcium-Ion/new-api-worker**, yang digunakan untuk menyembunyikan alamat IP asli server agregator saat menghubungi penyedia API hulu dengan menyebarkan skrip index.js secara gratis pada **Cloudflare Workers**. Dengan mengikat domain